

Designing system queries

Chris Richardson

Microservice architecture consultant and trainer

Founder of the original CloudFoundry.com

Author of POJOs in Action and Microservices Patterns

Founder of Eventuate.io

 @crichardson

chris@chrisrichardson.net

<http://adopt.microservices.io>

Agenda

- Overview of designing queries
- Using the API Composition pattern
- Using the CQRS pattern

Designing system queries

Query specification

Query Canvas

Subdomain definitions

Subdomain canvas	
Name	Restaurant Management
Responsibilities	- Manages information about restaurants - Knows restaurant name, location, description, opening hours, and menu
Collaborations	External address validation and geocoding service
Type	Supporting
Complexity	Simple
Key aggregates	Restaurant

Service definitions

Service

Subdomain A

Subdomain B

Service	Subdomain A	Subdomain B
createOrder()		
findOrder()		
updateOrder()		
rejectOrder()		
reserveCredit()		
approveOrder()		

Design system query

Generate candidate designs

Group subdomains

Design collaborations

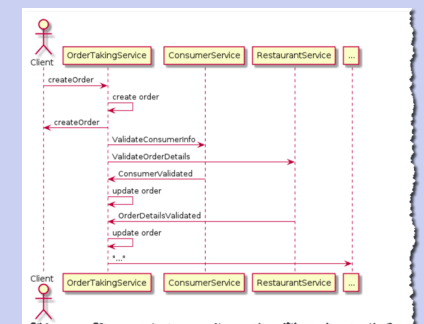
Evaluate candidates

Select candidate design

Query collaboration design

System command: createOrder()			
Initiating service: Order Service			
Steps			
Step No.	Participant	Transaction	Compensating Transaction
1	Order Service	createOrder()	rejectOrder()
2	Customer Service	reserveCredit()	-
3	Order Service	approveOrder()	-

Query:	findOrder()
Parameters:	TBD
Returns:	TBD
Strategy:	API Composition
Participants	
Order Service	findOrder()
Delivery Service	findDeliveryByOrderId(orderId)
Kitchen Service	findTicketByOrderId(orderId)



Service definitions

Service

Subdomain A

Subdomain B

Service	Subdomain A	Subdomain B
createOrder()		
findOrder()		
updateOrder()		
rejectOrder()		
reserveCredit()		
approveOrder()		

Let's imagine that you are implementing the findOrder() query

Service	Operation
Order Service	Status of order: line items
Kitchen Service	Status of ticket: ready for pickup
Delivery Service	Status of delivery: ETA
Accounting Service	Status of payment: total including tax

Traditional distributed query = ACID yet runtime and design-time coupling

findOrder()



```
BEGIN transaction
...
SELECT *
FROM ORDERS, TICKET, ...
WHERE ...
...
COMMIT transaction
```

Order Service

Database

Kitchen Service

Database

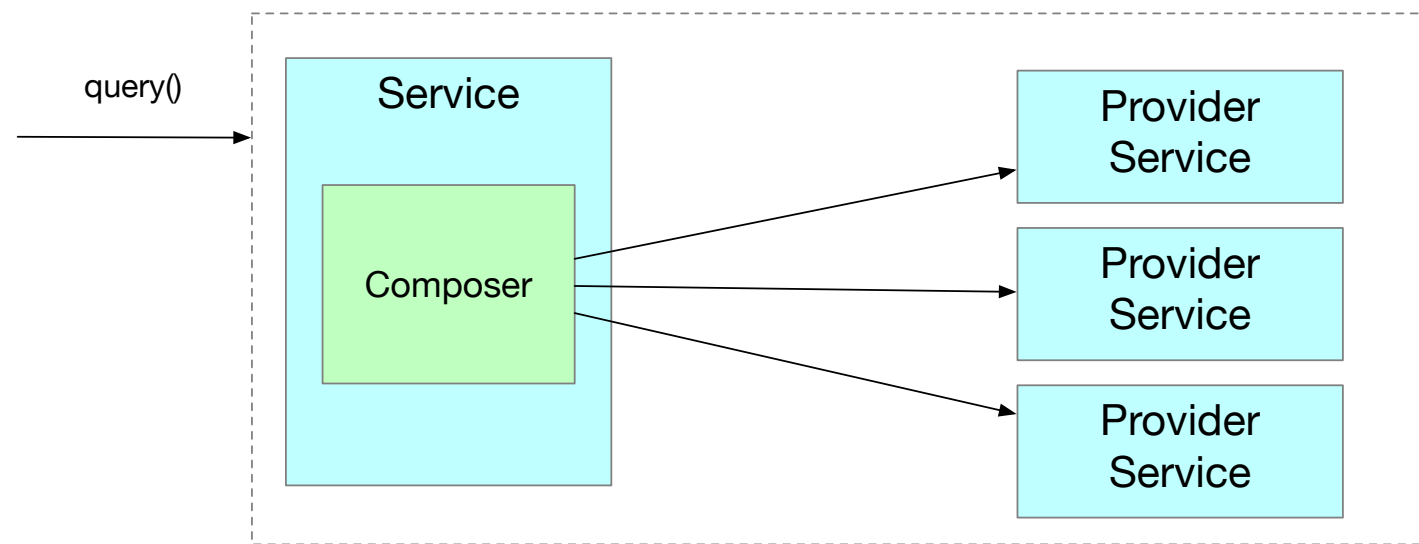
... Service

Database

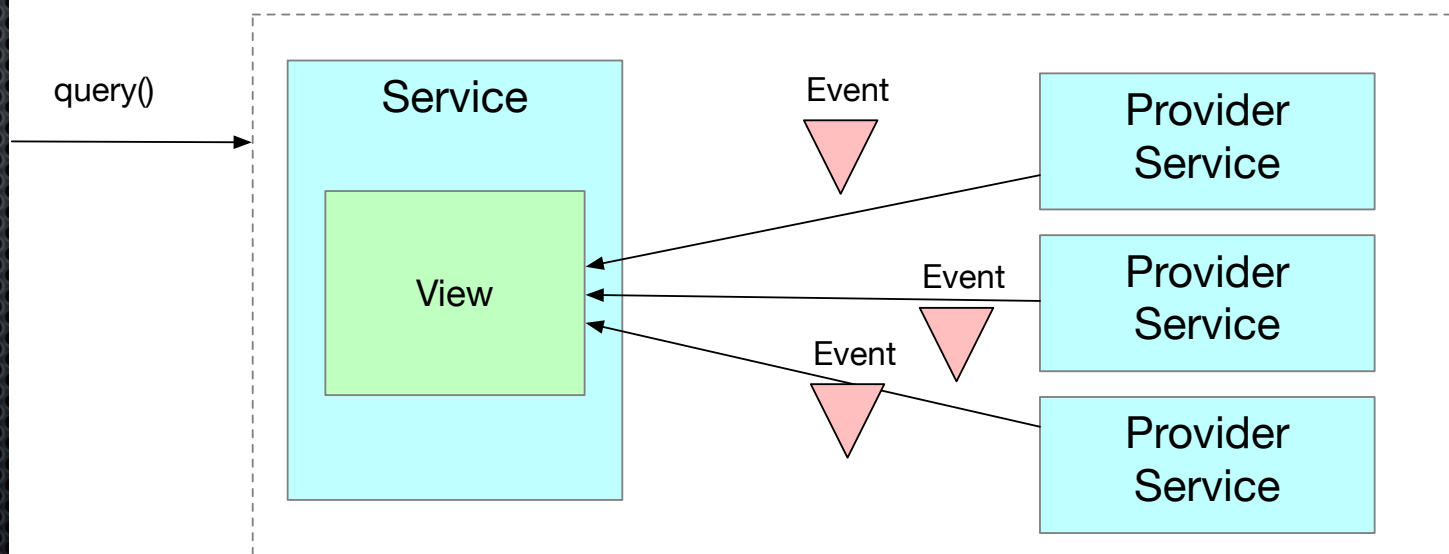
Distributed ACID transaction

Query patterns

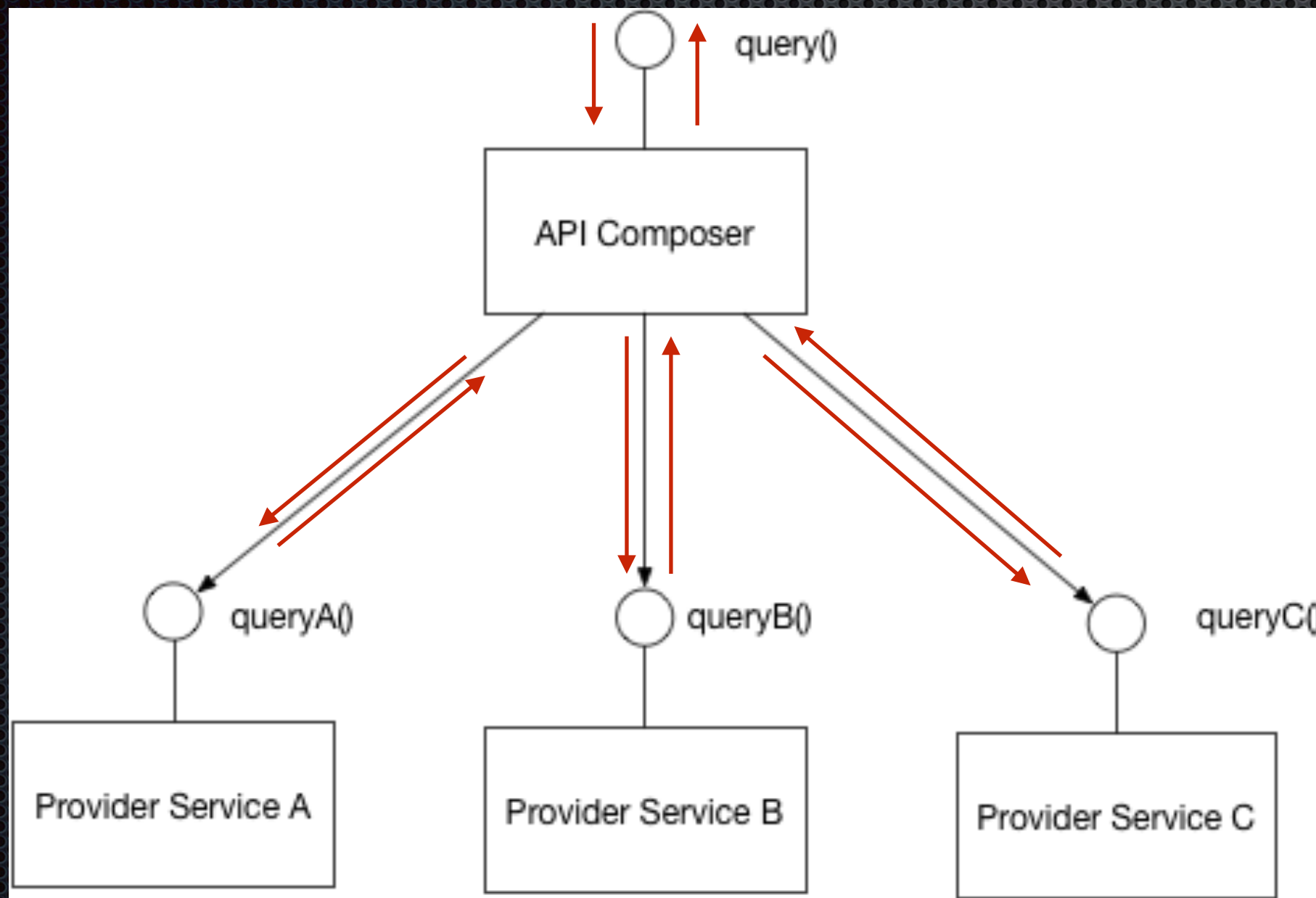
API Composition



CQRS

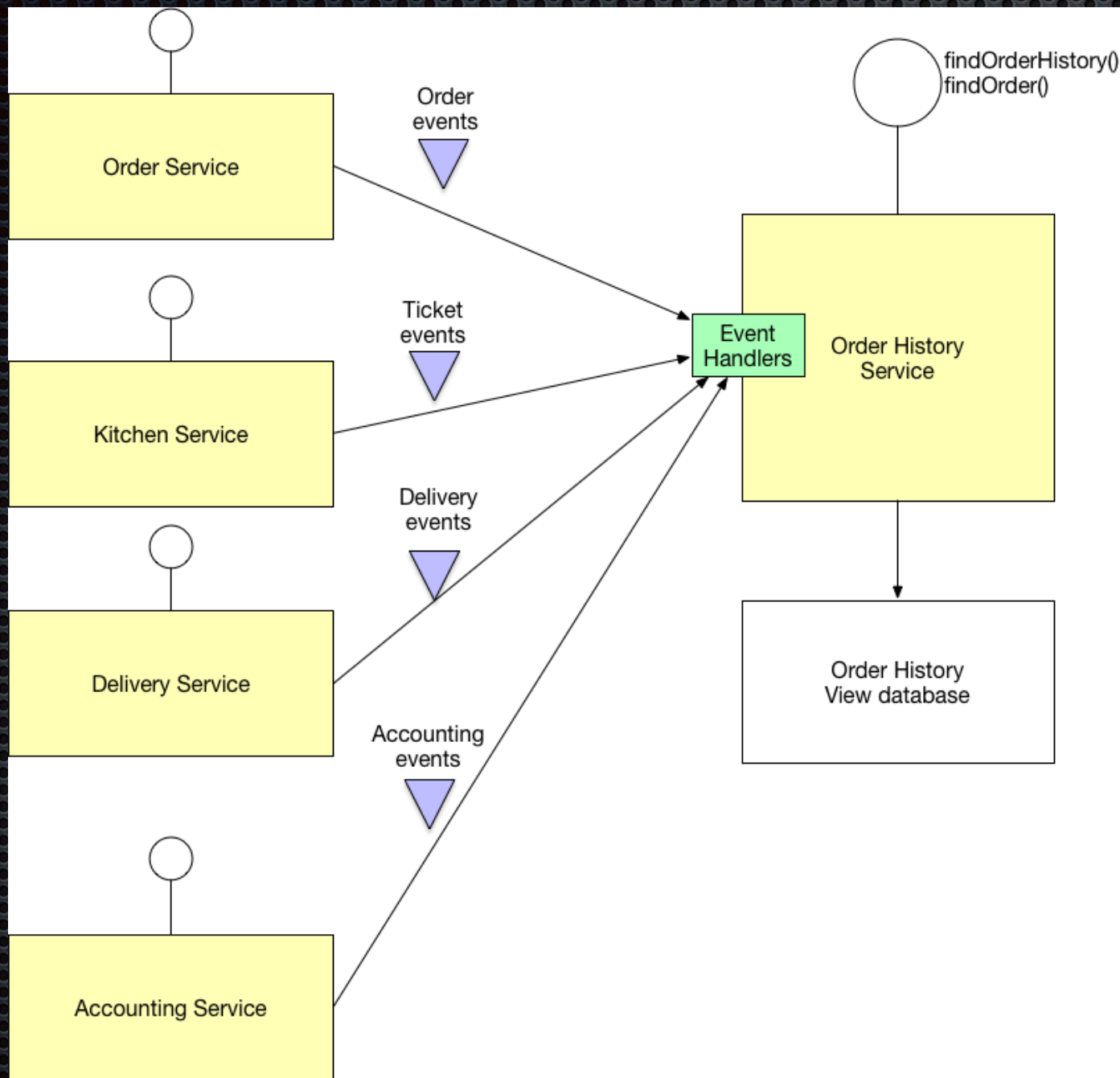


API Composition pattern



Simple
but sometimes
too inefficient

CQRS pattern



More flexible
but
more complex
and
eventually
consistent

<https://microservices.io/patterns/data/cqrs.html>

Many possible designs

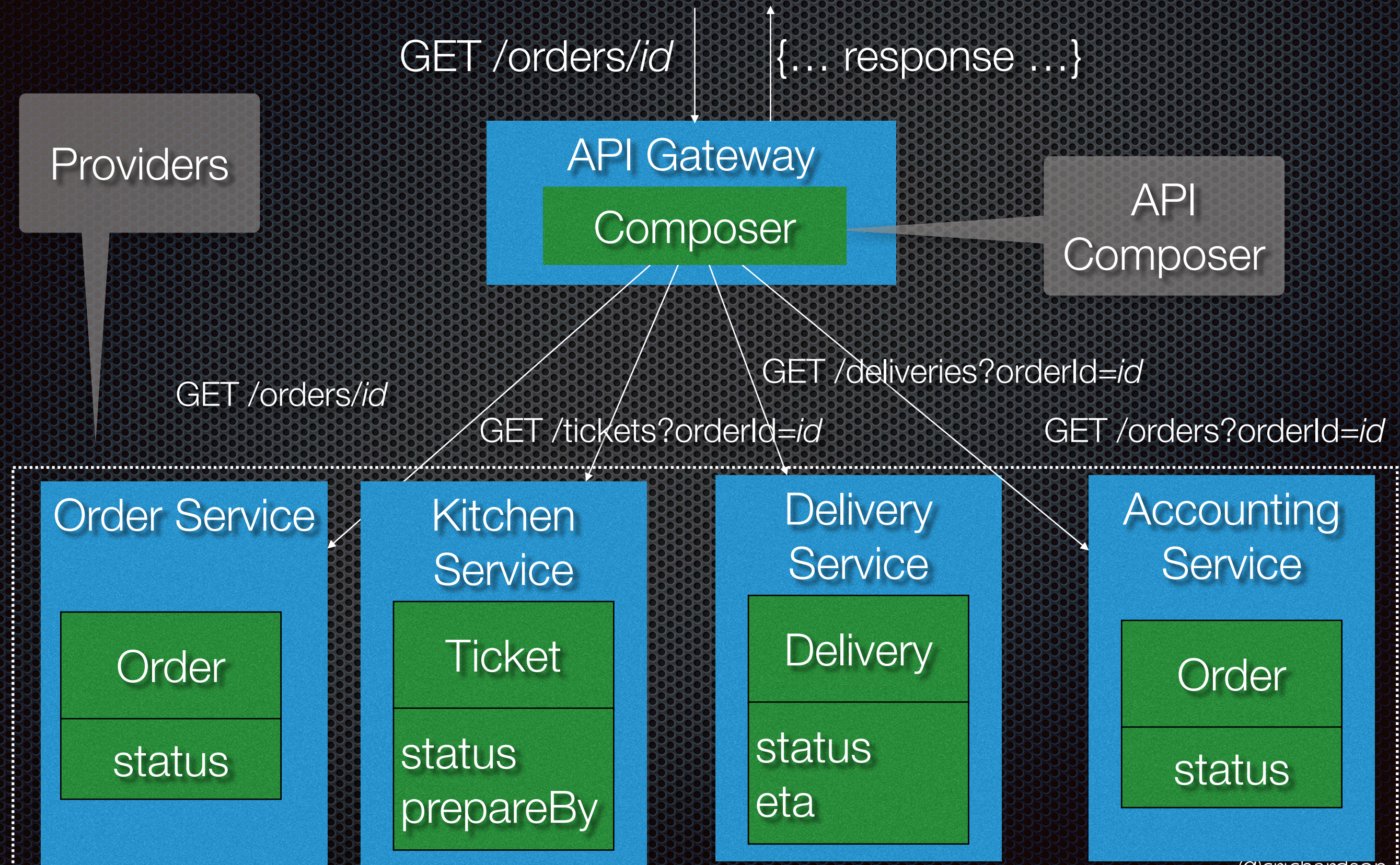
=>

evaluate dark energy/matter
forces and compare

Agenda

- Overview of designing queries
- Using the API Composition pattern
- Using the CQRS pattern

API Composition pattern



Benefits and drawbacks of API Composition

Dark energy repulsive forces

Support multiple technology stacks



Dark matter attractive forces

Simple interactions

? Possibility of many roundtrips

Prefer ACID over BASE

✗ Multiple queries - potential for inconsistencies

Minimize runtime coupling

✗ Use resiliency patterns (eg. Circuit breaker) to reduce impact but not always sufficient

Efficient inter-service communication

? Parallelization can reduce latency
Possibility of many roundtrips or large data transfers

Minimize design time coupling

✓ API-based collaboration
✗ Composer coupled to providers

Query consistency: read-your-writes 😊

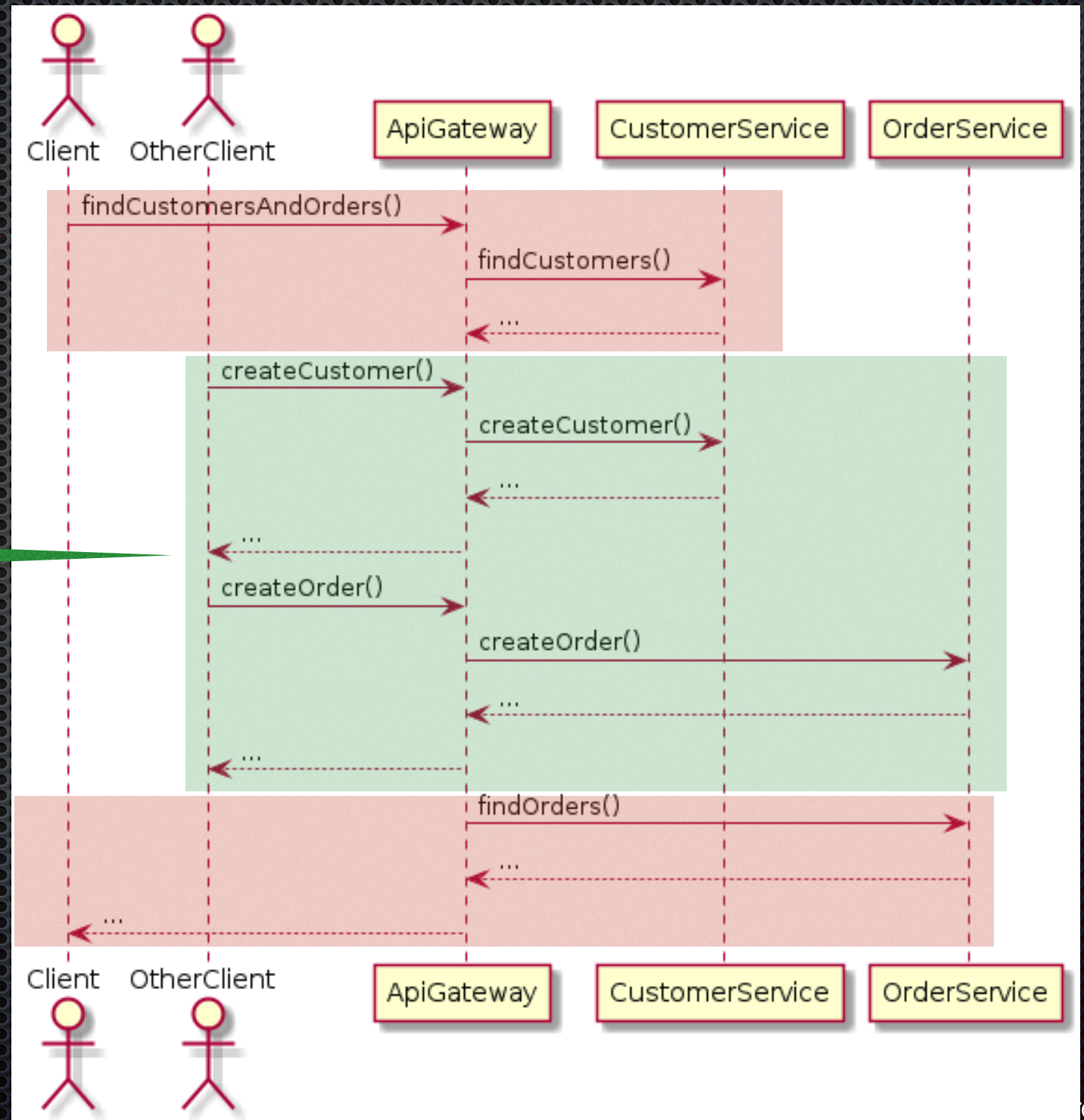
- ✦ A client can:
 - ✦ Execute a command
 - ✦ Immediately execute a query implemented using API Composition
 - ✦ The query will return the results of the command
- ✦ The CQRS pattern does not guarantee this

Query consistency: inconsistency across queries 🥹

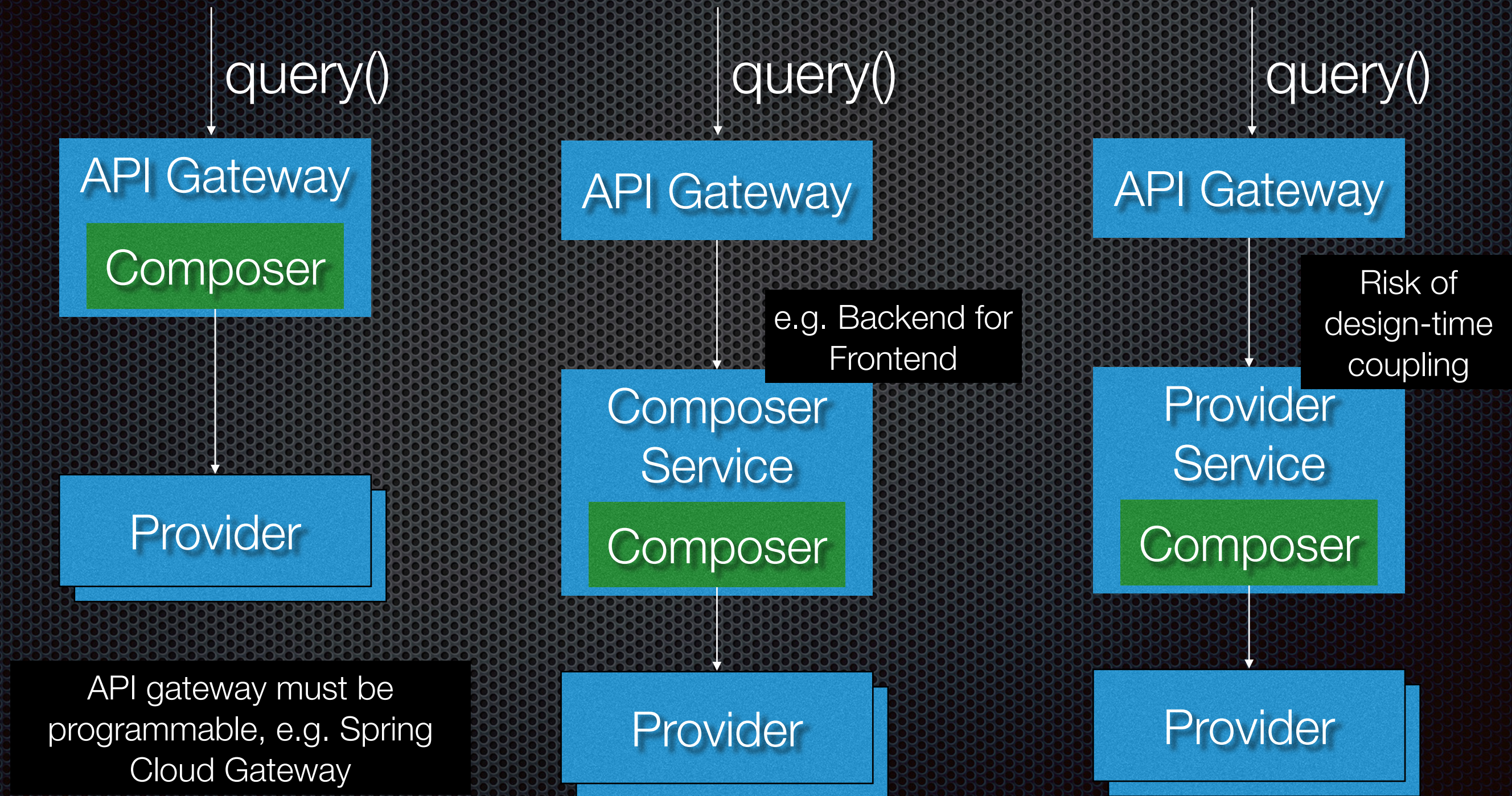
Retrieves
C1, C2

Creates C3
and Order

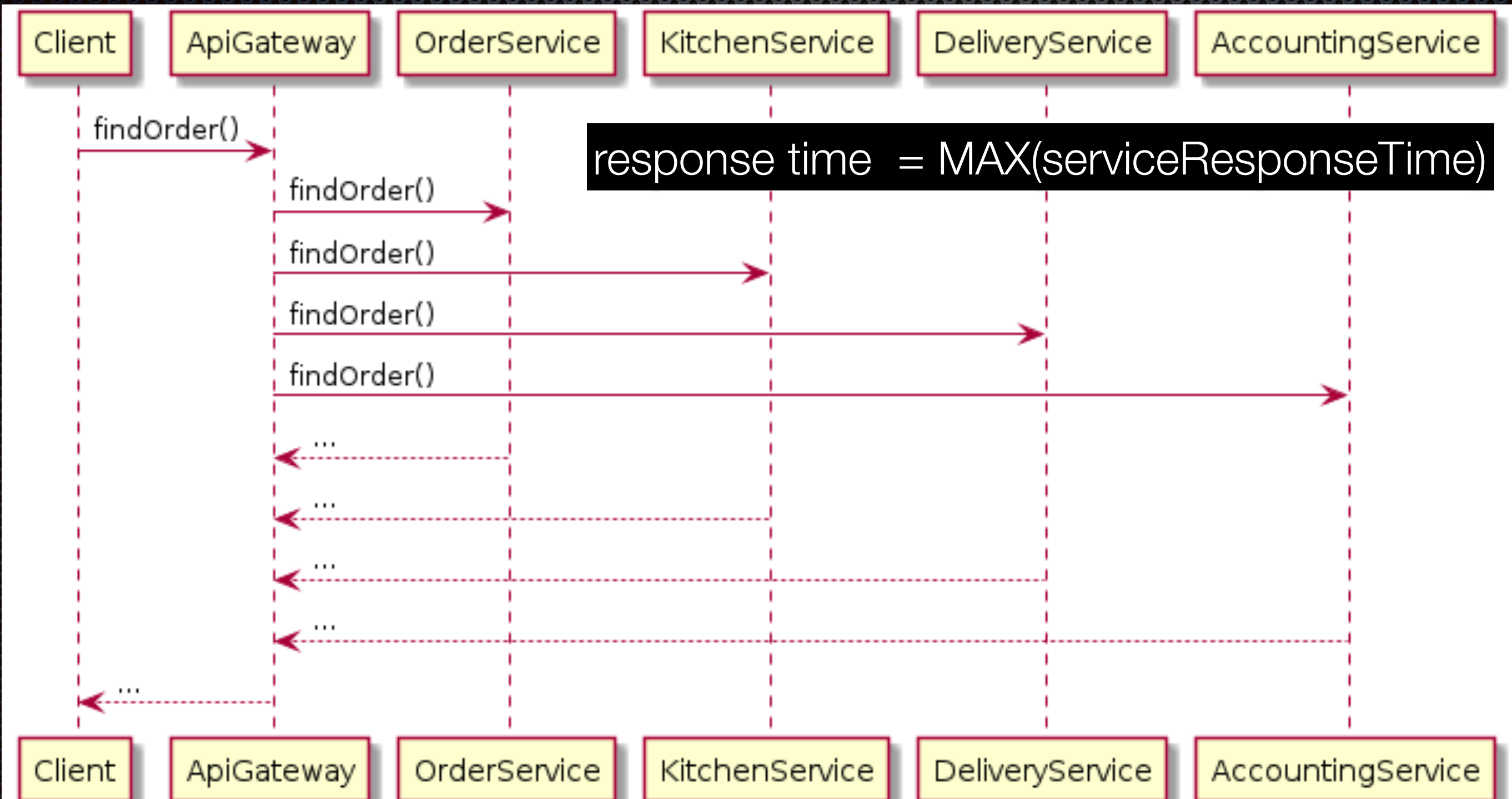
Returns
Order for C3



Applying the API Composition pattern: Determine API Composer location

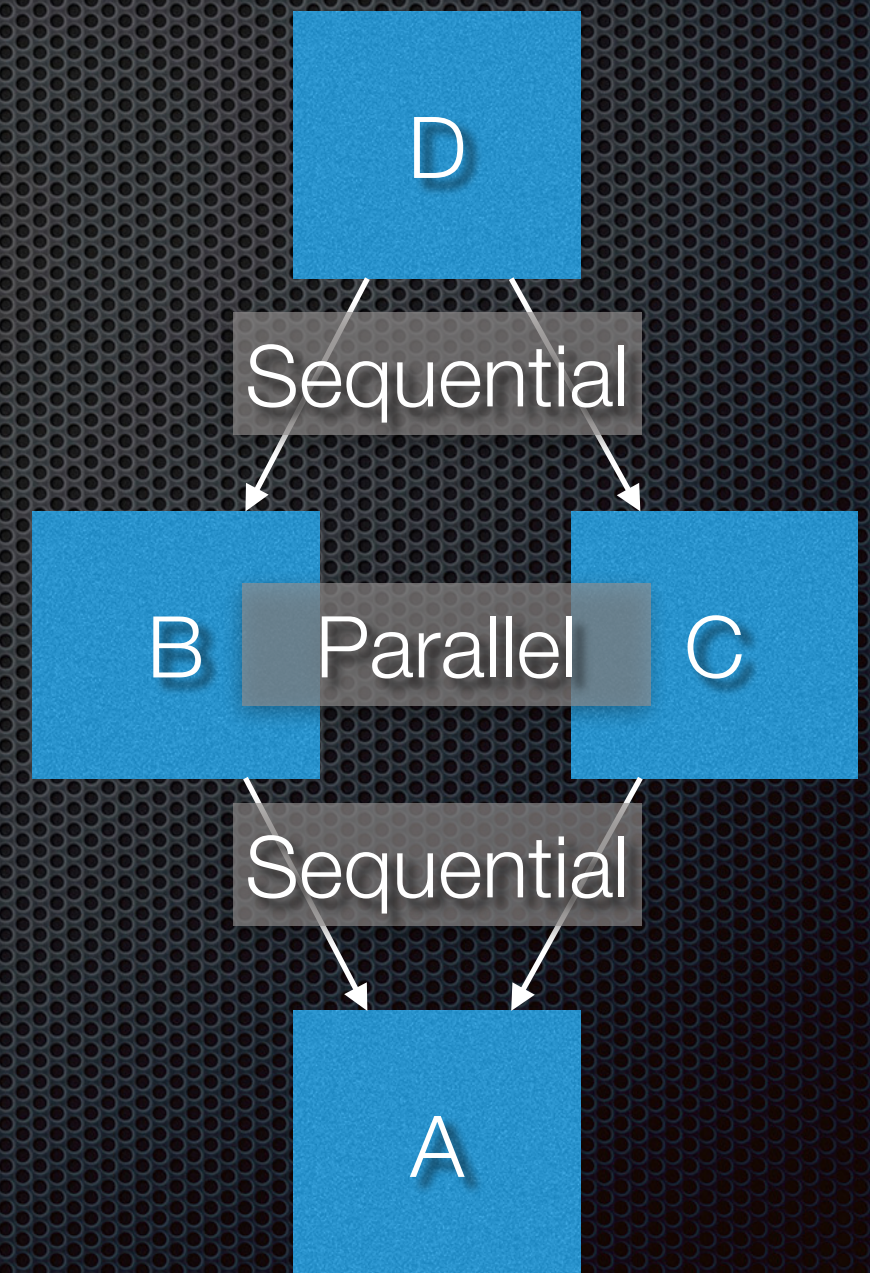


API composition implementation: parallelize



Data dependencies can reduce parallelism

- The arguments to one invocation = result of previous invocation $\Rightarrow$ invoke sequentially, e.g.
 1. Invoke Order Service to get orders
 2. Invoke Delivery Service to get delivery status of each Order
- In general: there is a dependency graph: a query cannot be executed until its dependencies are satisfied
- API Composition is a mix of sequential and parallel calls



Efficient, maintainable API composition = reactive code

```
public OrderHandlers(OrderServiceProxy orderService,
                    RestaurantOrderService restaurantOrderService,
                    DeliveryService deliveryService,
                    AccountingService accountingService) {...}

public Mono<ServerResponse> getOrderDetails(ServerRequest serverRequest) {
    String orderId = serverRequest.pathVariable(name: "orderId");

    Mono<OrderInfo> orderInfo = orderService.findOrderById(orderId);

    Mono<Optional<RestaurantOrderInfo>> restaurantOrderInfo = restaurantOrderService
        .findRestaurantOrderById(orderId)
        .map(Optional::of)
        .onErrorReturn(Optional.empty());

    Mono<Optional<DeliveryInfo>> deliveryInfo = deliveryService
        .findDeliveryById(orderId)
        .map(Optional::of)
        .onErrorReturn(Optional.empty());

    Mono<Optional<BillInfo>> billInfo = accountingService
        .findBillById(orderId)
        .map(Optional::of)
        .onErrorReturn(Optional.empty());

    Mono<Tuple4<OrderInfo, Optional<RestaurantOrderInfo>, Optional<DeliveryInfo>, Optional<BillInfo>>> combined =
        Mono.when(orderInfo, restaurantOrderInfo, deliveryInfo, billInfo);

    Mono<OrderDetails> orderDetails = combined.map(OrderDetails::makeOrderDetails);

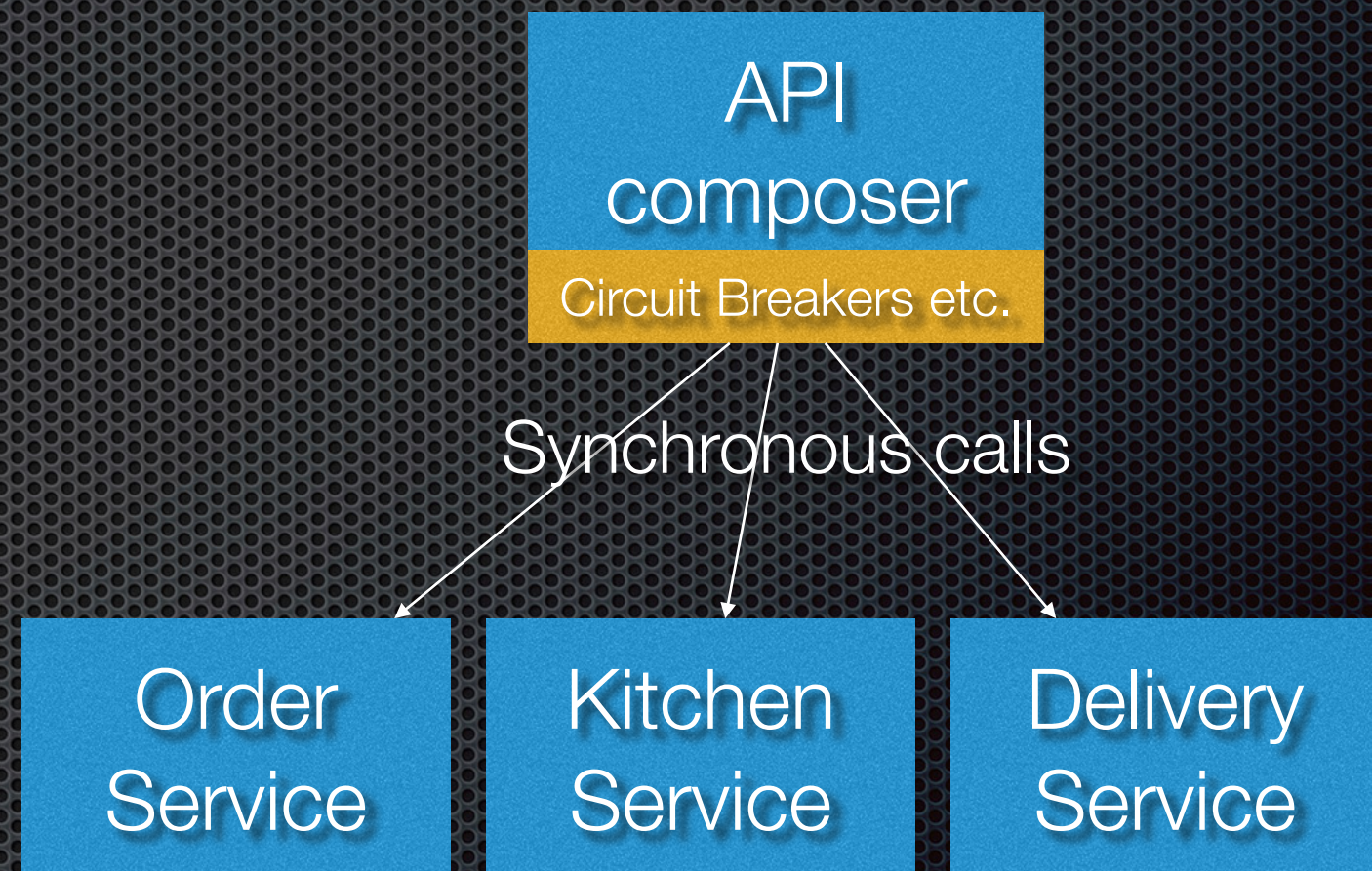
    return orderDetails.flatMap(od -> ServerResponse.ok()
        .contentType(MediaType.APPLICATION_JSON)
        .body(fromObject(od)));
}
```

An asynchronous call

Asynchronously combine
the result of the 4 calls

API composition implementation: resilience

- ✦ API composition = runtime coupling = risk of lower availability
- ✦ Use resiliency techniques: Timeouts, Retries, Circuit Breaker
- ✦ Use a fallback mechanism for each optional provider: empty, default, or cached data



Updated service definitions: API Composition

- ✦ Composition in API Gateway
 - ✦ Implement endpoint for system query
- ✦ Composition in Service
 - ✦ Update API Gateway to route to Composer
 - ✦ Add API query for the system query
- ✦ Providers services
 - ✦ Implement API query operations

API Composition works well in many situations

BUT

Sometimes it's too inefficient and runtime
coupled

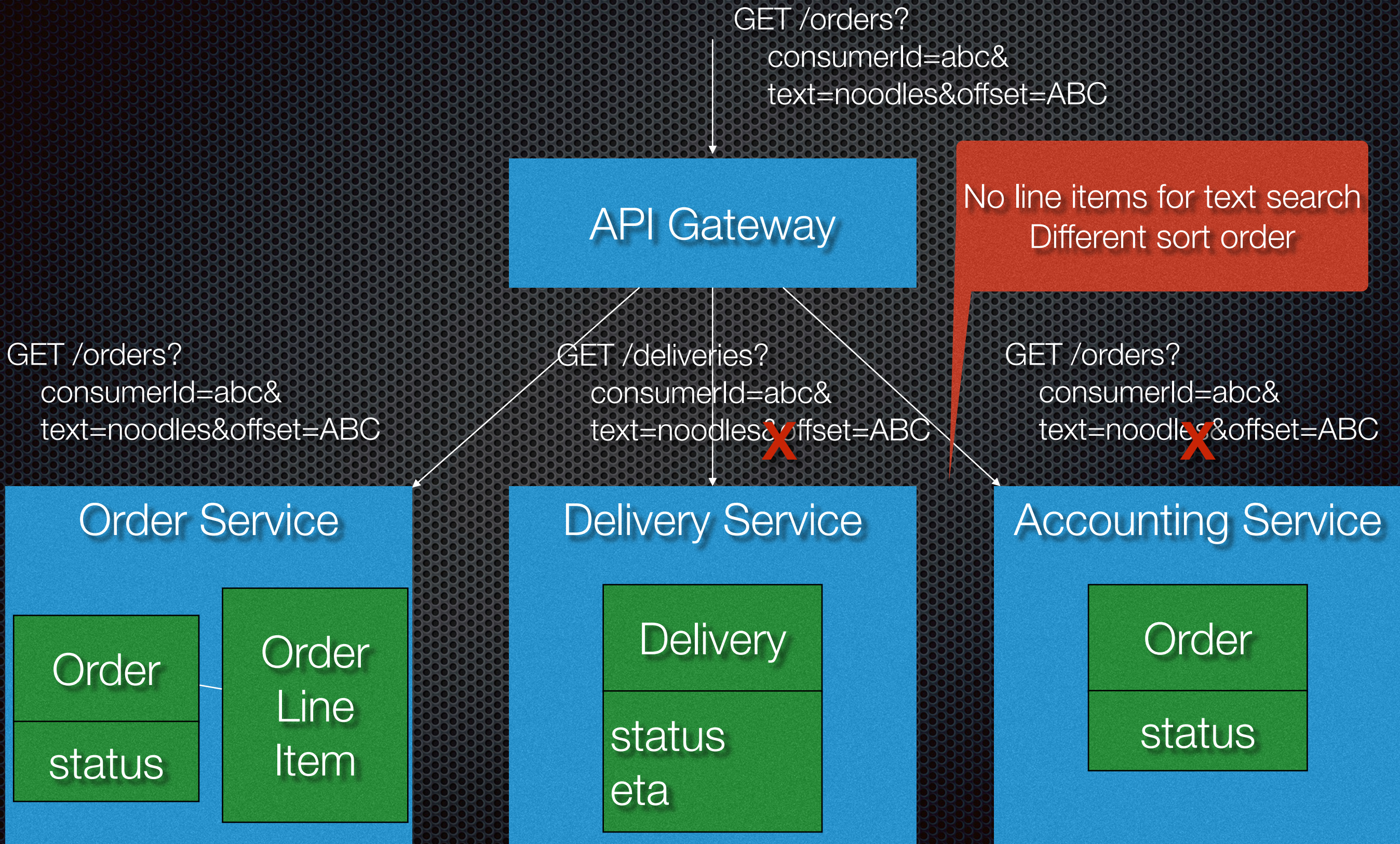
Excessive runtime coupling

- Resiliency patterns (e.g. Circuit breaker) reduces coupling to optional provider services

BUT

- Composer is still coupled to mandatory providers

Inefficient API composition



... Inefficient API composition

- Query Order Service to retrieve page of Orders containing noodles
- For each Order:
 - Query Delivery Service
 - Query Accounting Service
- Potentially optimize with batch APIs: reduce $1+N$ calls to 3

Even worse

Orders that contain noodles
and
have been delivered

... Inefficient API composition

- Query Order Service to retrieve page of Orders containing noodles
- For each Order:
 - Query Delivery Service to find delivered Orders
 - Query Accounting Service
- Potentially optimize with batch APIs: but potentially many roundtrips to get a page of matching orders
- Inefficient with a high response time

If API composition is too
inefficient or too runtime coupled



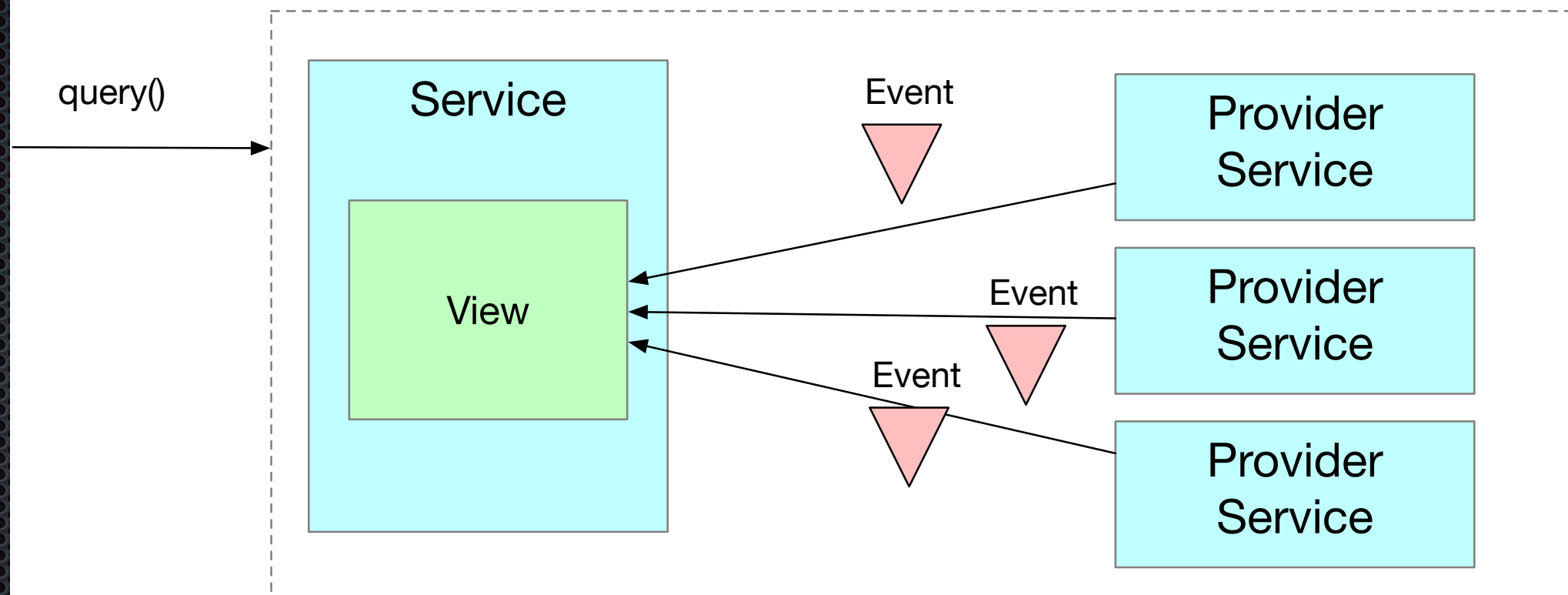
Use the CQRS pattern
OR

Reconsider your architecture

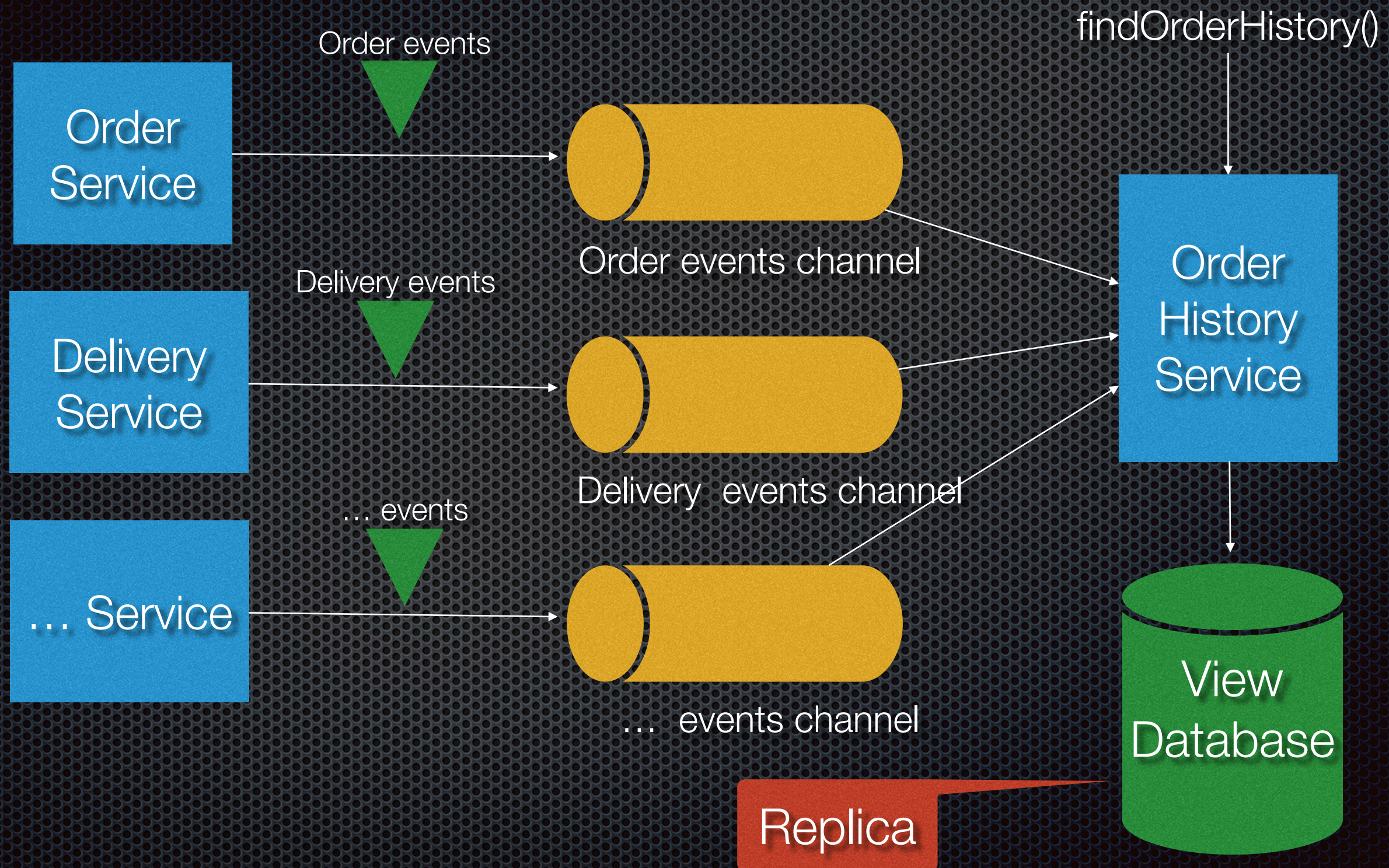
Agenda

- Overview of designing queries
- Using the API Composition pattern
- Using the CQRS pattern

CQRS



Implementing a query using CQRS



Benefits and drawbacks of CQRS

Dark energy repulsive forces

Support multiple technology stacks

✓ View DB different than Provider DBs

Dark matter attractive forces

Simple interactions

✓ Query is local

✗ Other cmds must publish events

Efficient inter-service communication

✓ Query is self-contained and optimized

✗ Cost of replication - storage + events

Prefer ACID over BASE

✗ Out of order events => replication lag

✗ Out of order events => Inconsistencies

Minimize runtime coupling

✓ Query is self-contained + events

Minimize design time coupling

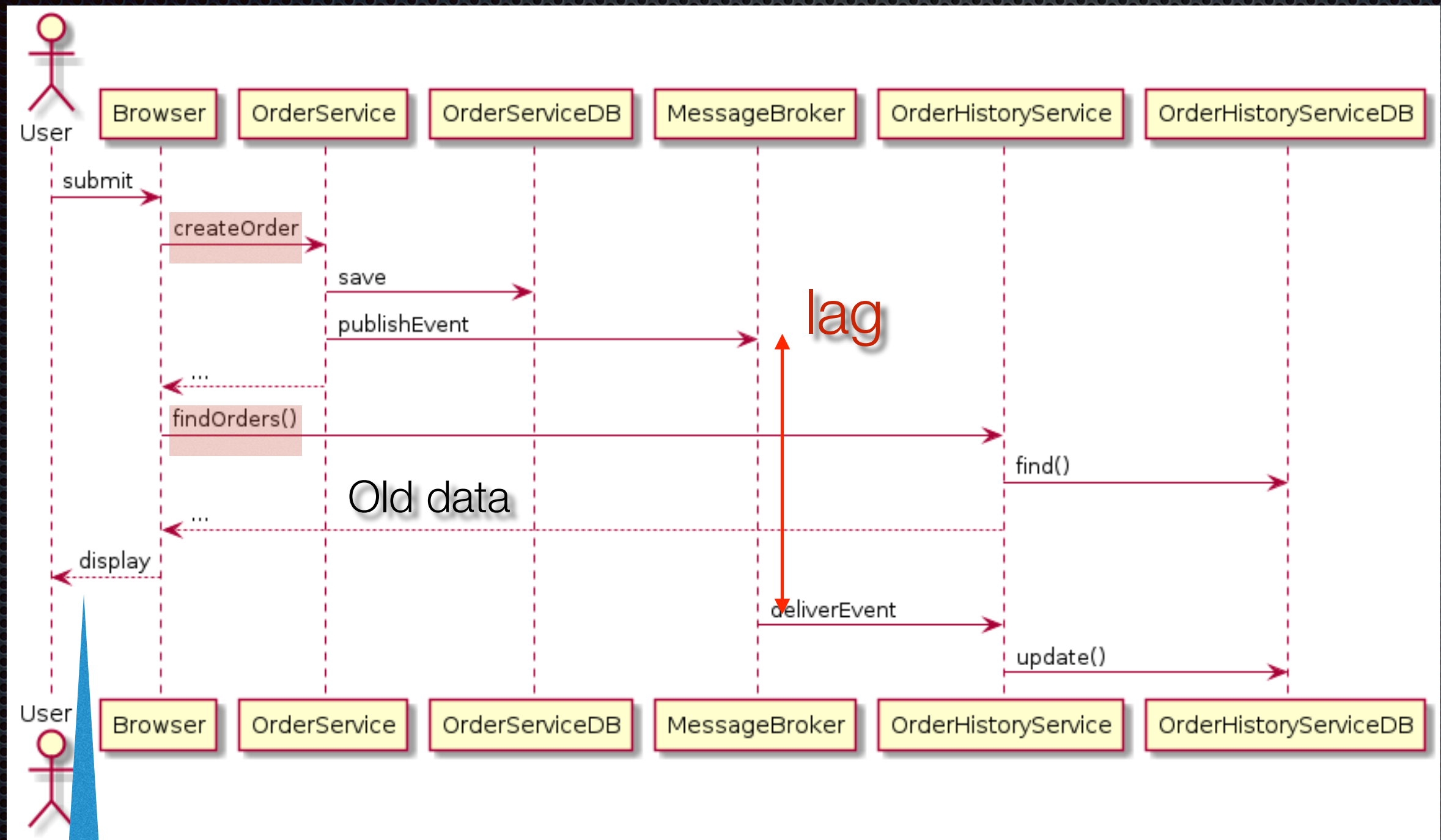
✓ API-based collaboration

✗ Risk of view being tightly coupled to providers

Handling CQRS inconsistency issues

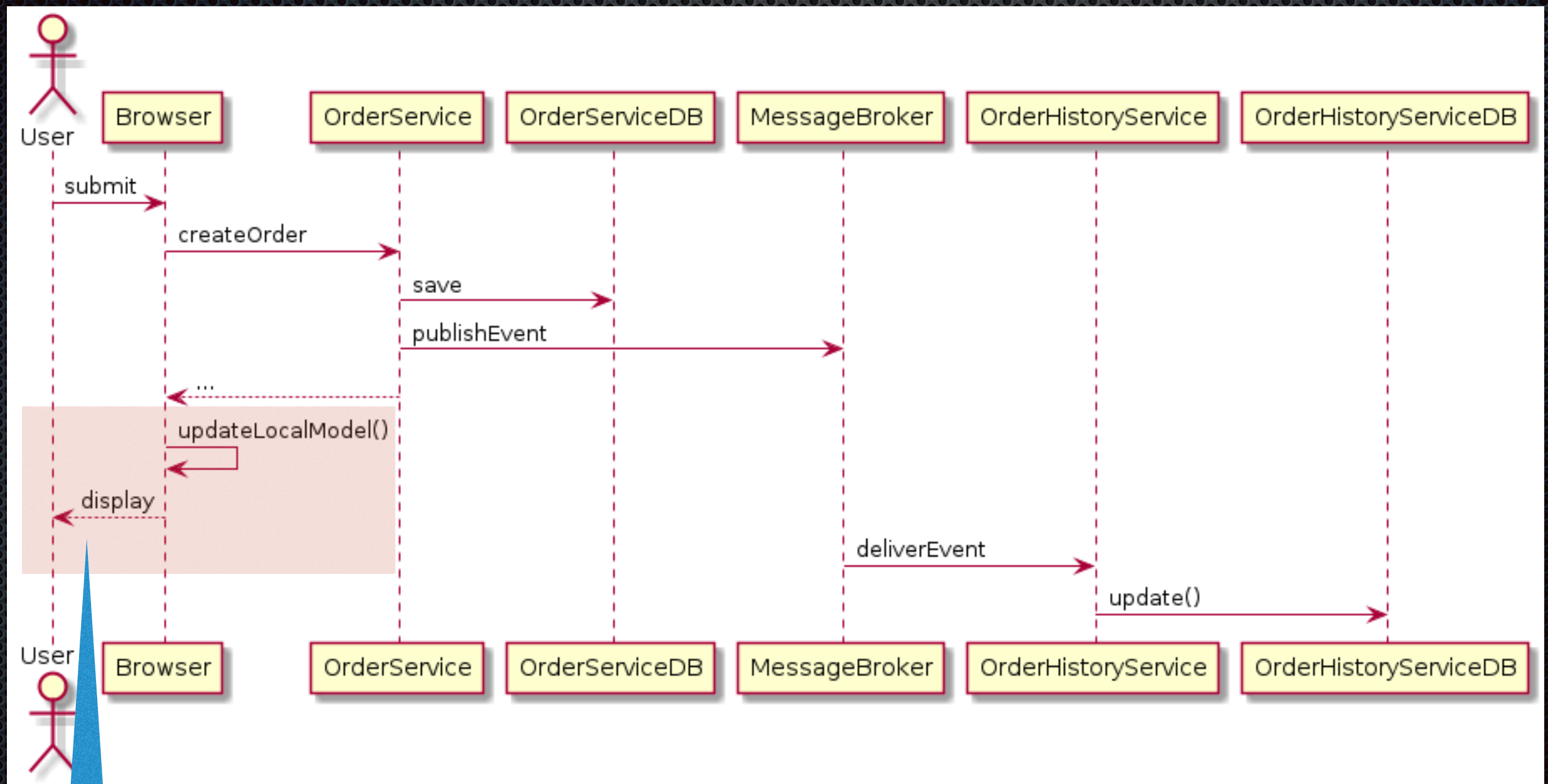
- Handling replication lag
 - Lag between updating command side and CQRS view
 - Risk of showing stale data to user => confusion
- Out of order event delivery
 - Events published by different aggregates are unordered
 - e.g. OrderCreated event arrives between CustomerCreated event

The problem of replication lag



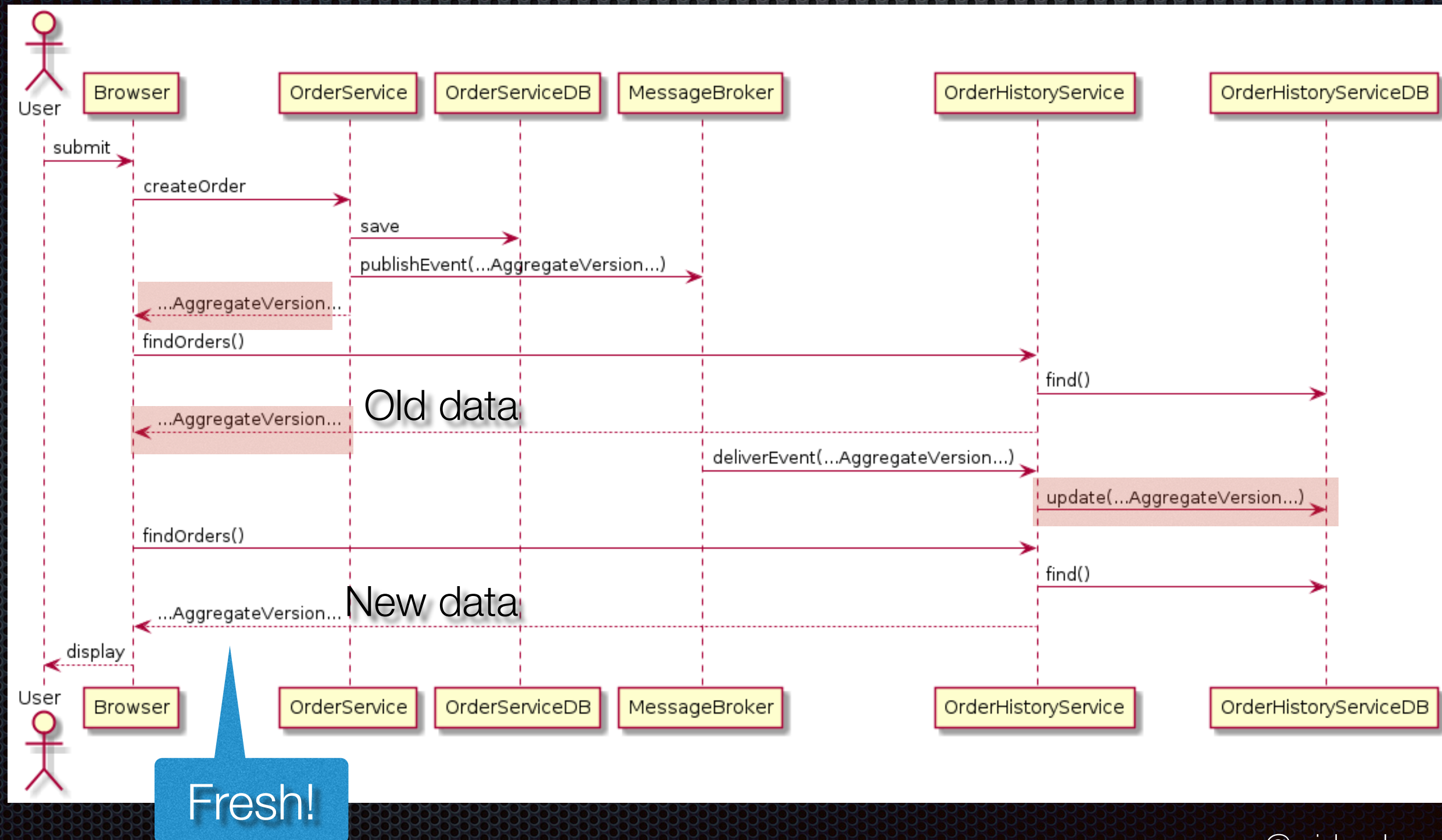
Stale!

Solution: update browser-side model



Fresh!

Solution: track aggregate version



Out of order delivery

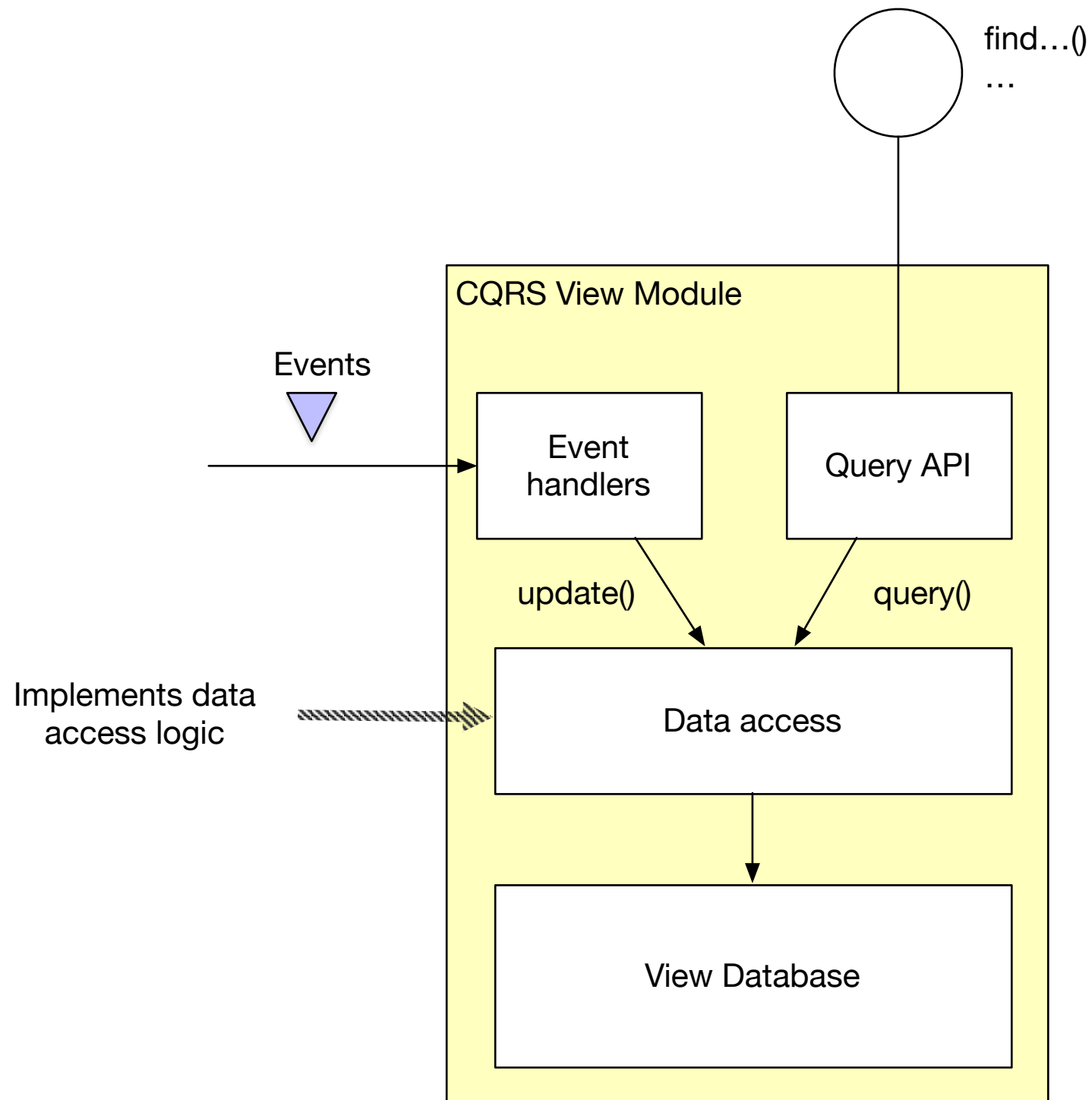
- Client:
 - creates Customer -> CustomerCreated event
 - creates Order -> OrderCreated event
- CQRS view might receive OrderCreated before CustomerCreated
 - e.g. published to different Kafka topics = no ordering guarantee
- CQRS view updating logic cannot assume ordering
- Query code might retrieve/return incomplete data

Implement queries that span
services
using CQRS
when
API Composition
is inefficient

Updated service definitions: CQRS

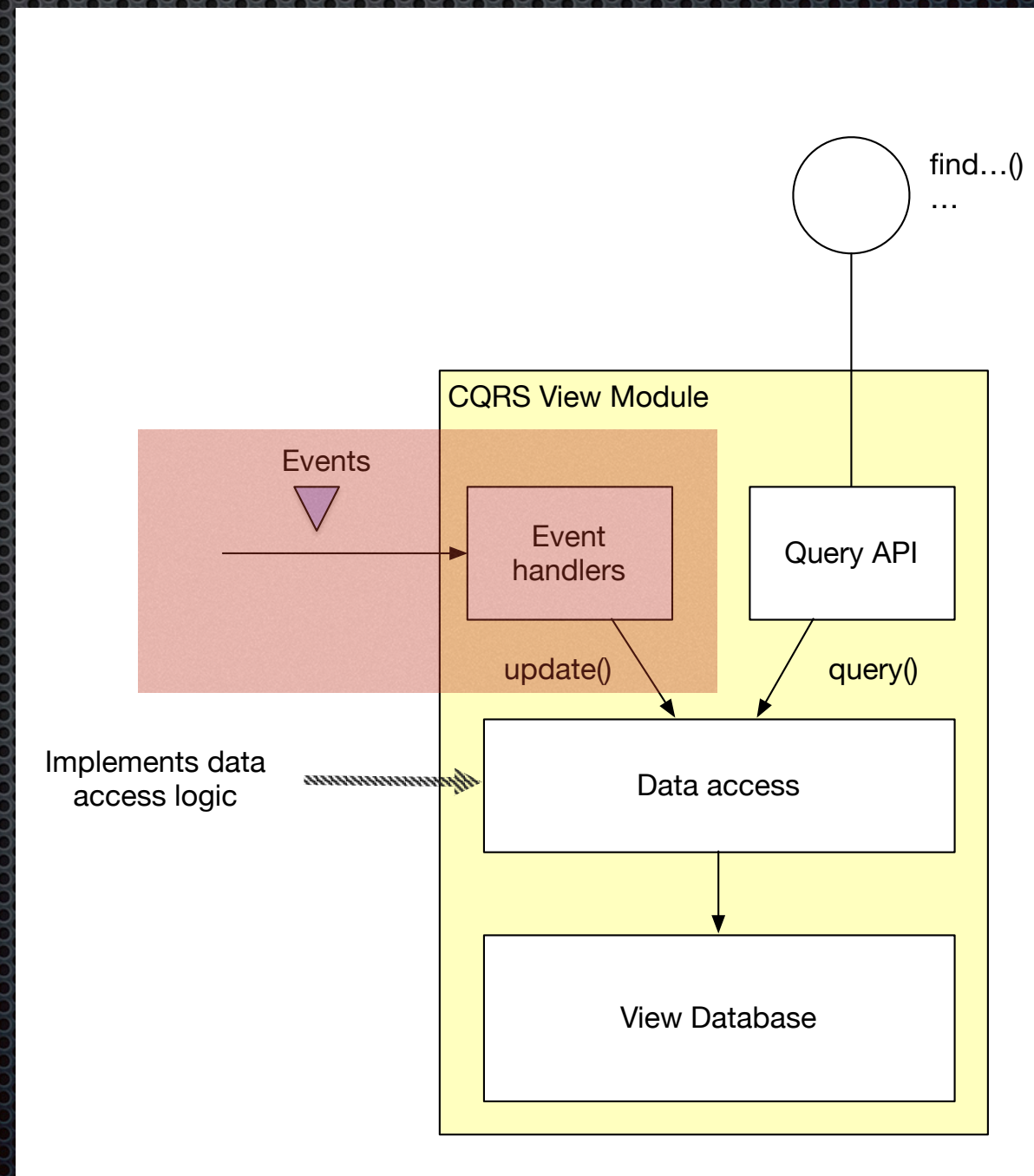
- Update API Gateway to route system query to CQRS service
- CQRS service
 - Implement event handlers
- Provider services
 - Publish required events

View module design



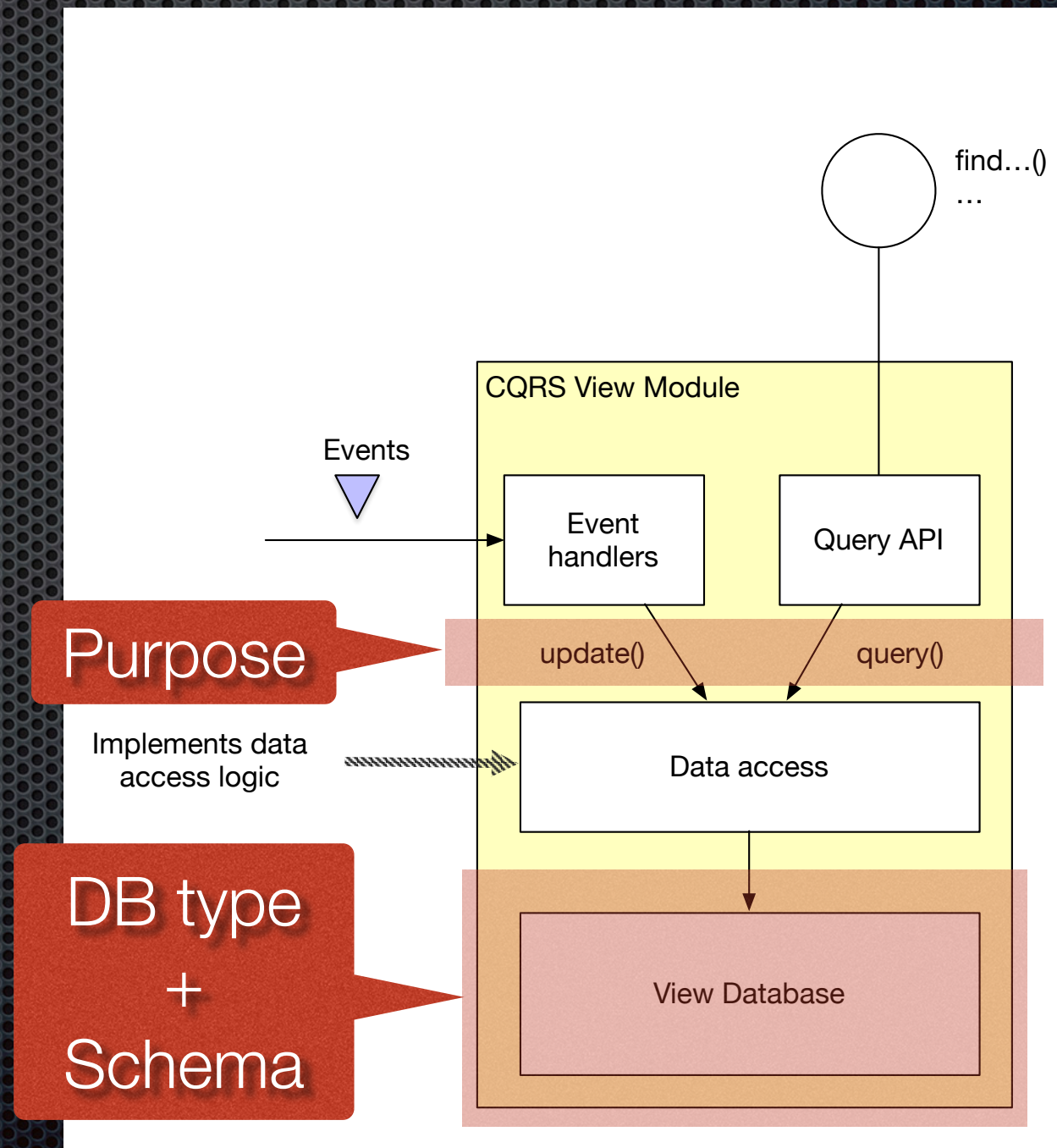
Design events and event handlers

- ✦ Design events that communicate changes made to business entities
- ✦ Write the event handlers that update the view



Select database and design schema

- Narrow scope: one or more find...() and update...() operations
- Issues: performance, scalability, technology governance, ...
- NoSQL is often a good choice
 - Leverage the benefits: scalability, rich data model, ...
 - Avoid the drawbacks: limited transactions, queries, ...



Select service to contain CQRS view module

- Options:

- Dedicated CQRS service
- Embedded in existing service

- Issues:

- Simplicity - components and interactions
- Minimizing design coupling
- Cost efficient scaling
- ...

Which service?

Service

CQRS view module

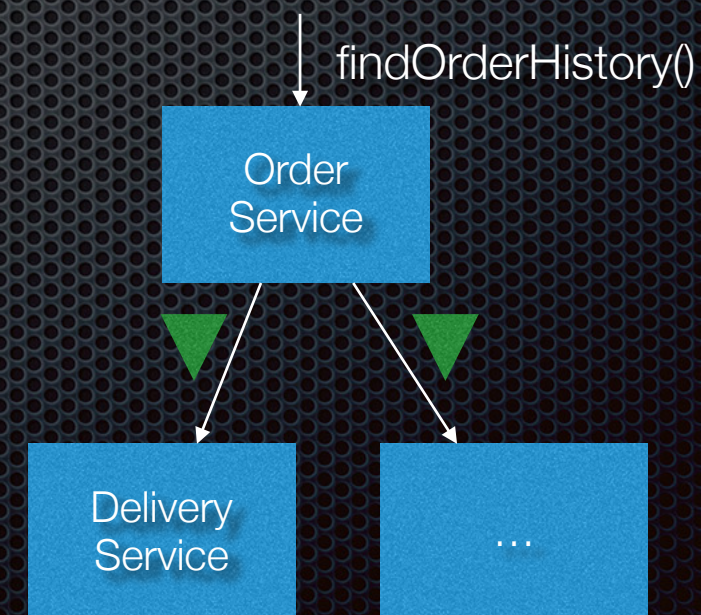
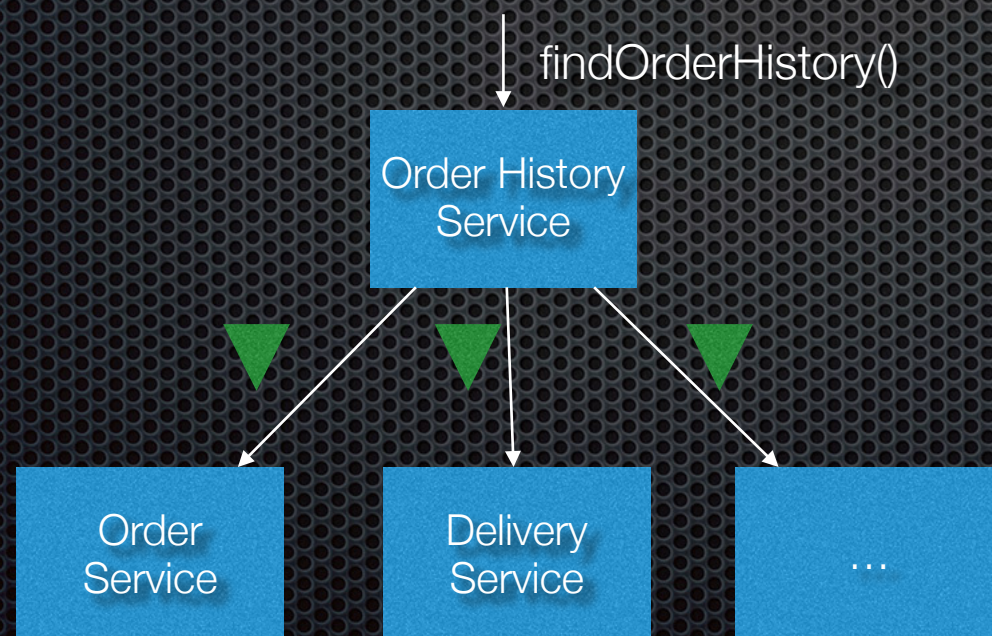
For example: findOrderHistory()

Order History Service

-  Simpler services
-  Improved scalability
-  More complex interactions
- ...

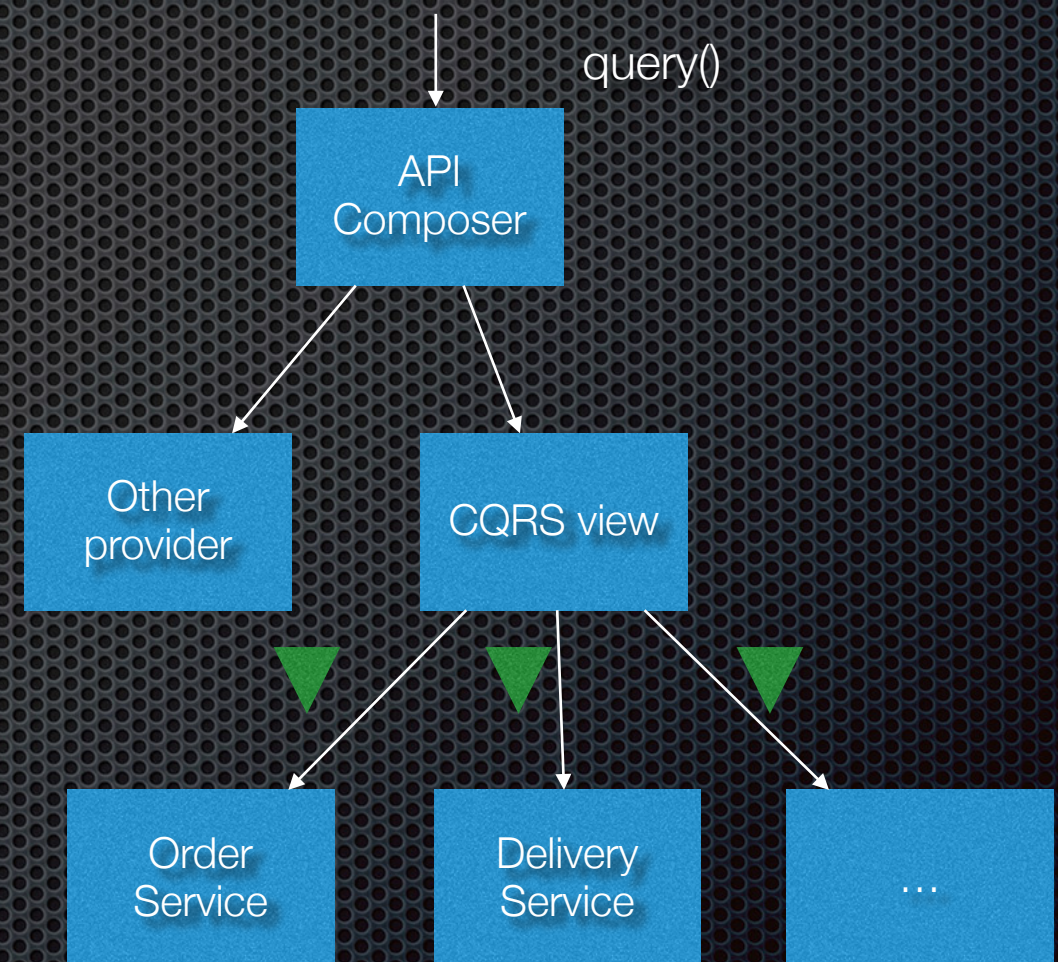
Order Service

-  Simpler interactions
-  More complex Order Service
-  Design-time coupled to providers
-  Increased load
- ...



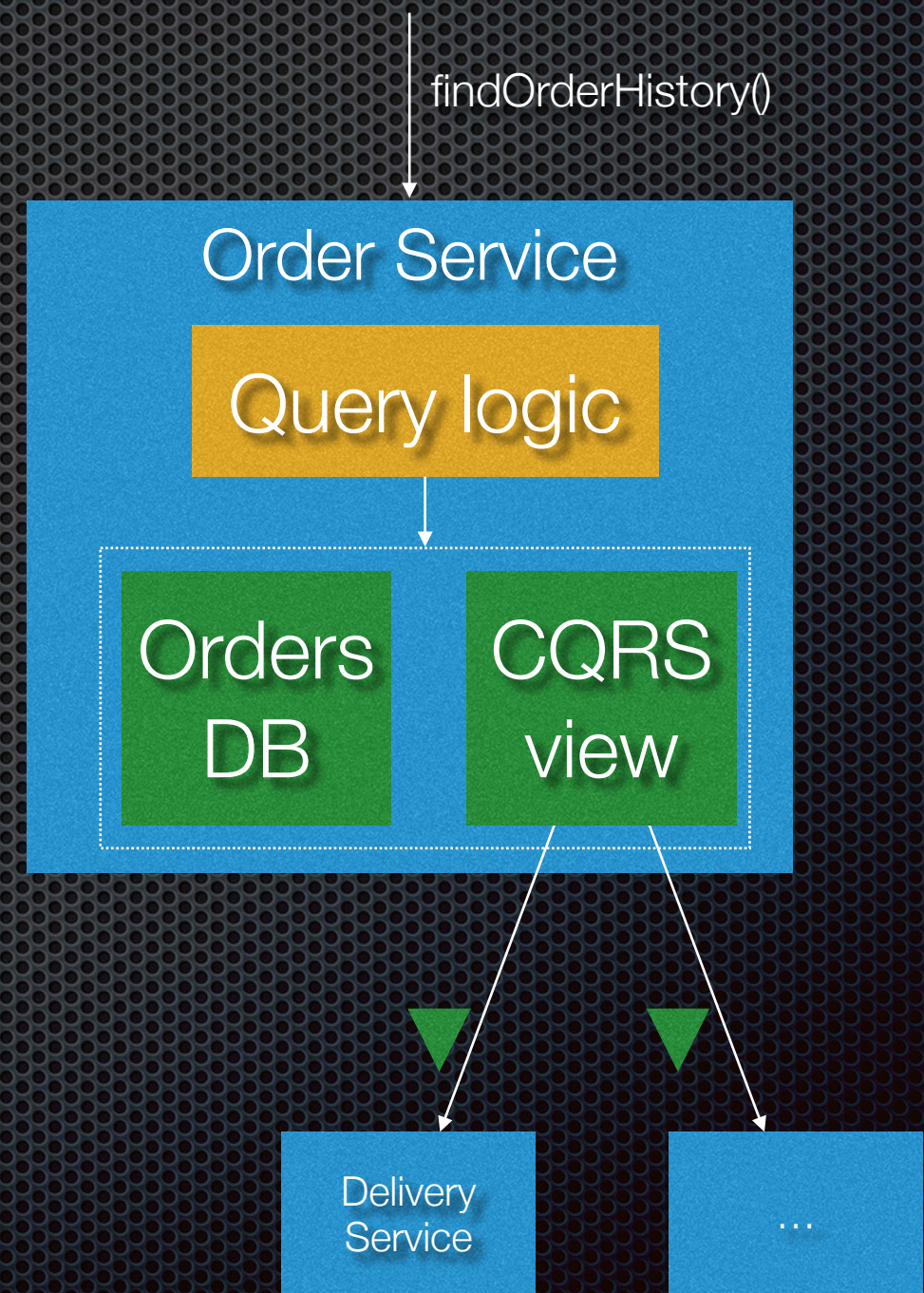
Variations of CQRS: API Composition + CQRS

- An API Composition provider can be a CQRS view
- Avoids replicating all data
- Potentially more efficient
- Requires more human query planning!

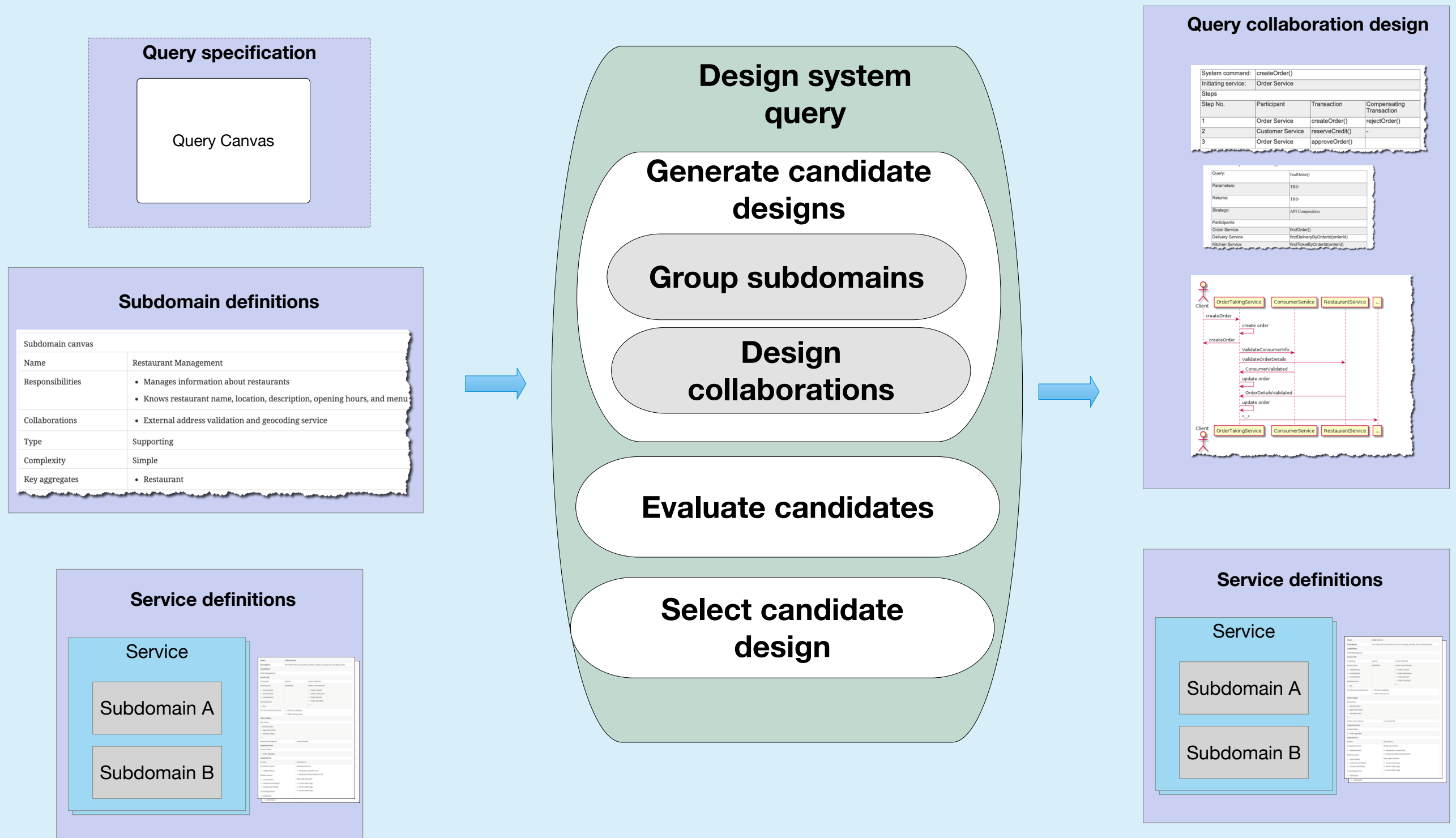


Variations of CQRS: Join with CQRS view

- ✦ Join source of truth DB with CQRS view
- ✦ Potentially more efficient



Summary: process



Summary: deliverables

Service

Subdomain A

Subdomain B

Service canvas

Name:	Order Service	
Description:		
Responsibilities	• Manages orders	
Subdomains	• Order management	
Service API		
Commands	Queries	Events Published
• createOrder()	• findOrder()	• OrderCreated
Non-functional requirements		
Dependencies		
Invokes	Subscribes to	
	Restaurant Service: • Restaurant Created • Restaurant Updated • Restaurant ...	

System query design canvas

Name: findOrder()

Entry point:

Service: API Gateway

Operation:

Collaboration

API Composition

CQRS

API composition canvas

Query: findOrder(param1, param2,)

Parameters: TBD

Returns: TBD

Strategy: API Composition

Composer: API Gateway

Providers

Order Service

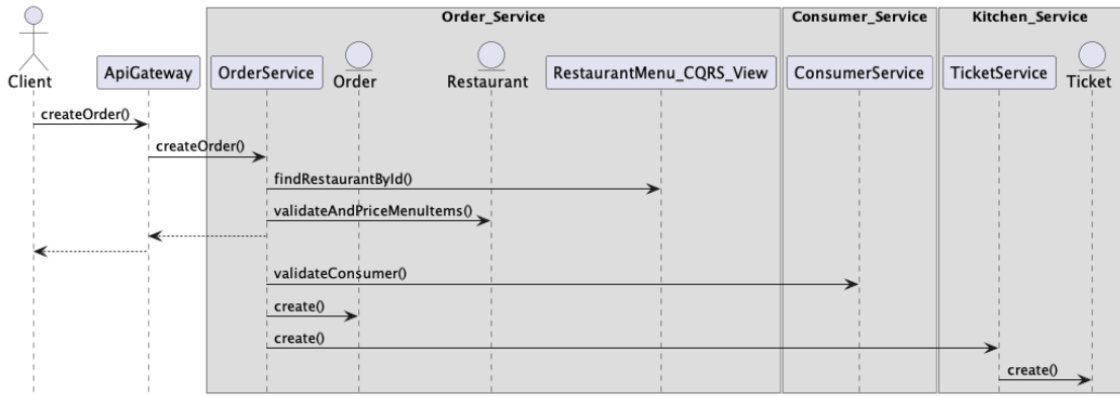
Delivery Service

Kitchen Service

Delivery Service

...

Query:	findOrder()
Parameters:	TBD
Returns:	TBD
Strategy:	CQRS
View Service:	Order History Service
View design	
Database type:	TBD
Schema design:	TBD
Command-side services	
Service name	Events
Order Service	• Order Created • Order Canceled
Delivery Service	• Delivery scheduled • Picked up



 [@crichardson](https://twitter.com/crichardson) chris@chrisrichardson.net

Questions?

<http://adopt.microservices.io>

About

Chapter 7. Kata #5: Design system queries

Design the services and queries and document them using the templates in [03-02-designing-operations-and-services](#)

Query specification

- ✦ As a group:
 - ✦ For each query:
 - ✦ Create candidate operation realizations
 - ✦ Evaluate using dark energy/matter
 - ✦ Select the best candidates
- ✦ In pairs: create sequence diagram for assigned queries